UNIVERSIDAD DE LAS AMÉRICAS PUEBLA Departamento de

Computación, Electrónica y Mecatrónica

Materia: LRT-2042 Electrónica Integrada

Profesor: Dr. José Luis Vázquez González

Periodo: Primavera 2026

Actividad 06: Implementación de un Módulo de Control de Acceso con PIC18F45K22

Luis Alexis Calvillo Deveze

Descripción del funcionamiento

El presente proyecto consiste en la simulación de un sistema básico de control de acceso utilizando el microcontrolador PIC18F45K22. El sistema opera bajo tres estados principales controlados por interruptores físicos:

- **Sistema Deshabilitado:** Cuando el interruptor general (SW_ENABLE) está en nivel lógico bajo (0), el sistema se bloquea, encendiendo permanentemente el LED rojo para indicar acceso denegado. En este estado, el botón de validación no registra códigos, aunque el display mantiene su funcionamiento de visualización.
- **Modo Usuario:** Al habilitar el sistema y colocar el interruptor de administración (SW_ADMIN) en 0, el usuario puede ingresar un código numérico del 0 al 9 utilizando el botón de incremento (BTN_UP). Si el usuario presiona el botón de validación (BTN_OK) y el número en el display coincide con el código almacenado en la memoria RAM, se enciende un LED verde durante 2 segundos indicando el acceso permitido. Si el código es incorrecto, el LED rojo se enciende por 2 segundos. Existe también un botón de borrado (BTN_CLR) que regresa el contador a 0.
- **Modo Administrador:** Si se habilita el SW_ADMIN (nivel alto, 1), el sistema entra en modo de configuración. El usuario selecciona un número con BTN_UP y, al presionar BTN_OK, este nuevo valor sobrescribe el código de acceso guardado en la memoria. Para confirmar la acción, el LED verde parpadea dos veces de manera secuencial.

Asignación de pines

Para la implementación en el PIC18F45K22, se distribuyeron las señales digitales de la siguiente manera:

Entradas (Puerto B - Configuradas con resistencias Pull-Down):

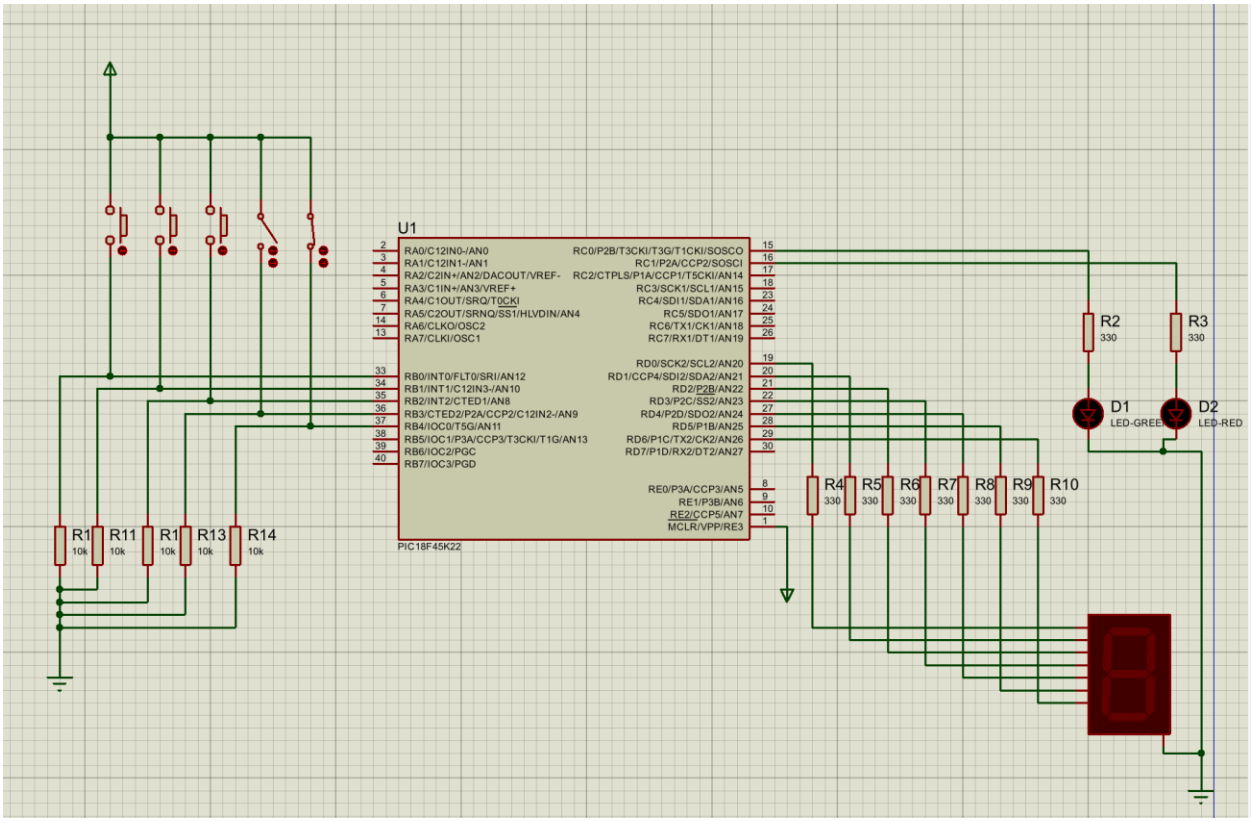
- RB0 -> BTN_UP (Pulsador para incrementar dígito)
- RB1 -> BTN_OK (Pulsador para confirmar código)
- RB2 -> BTN_CLR (Pulsador para reiniciar dígito a 0)
- RB3 -> SW_ADMIN (Interruptor selector de Modo Usuario/Admin)

- RB4 -> SW_ENABLE (Interruptor de habilitación del sistema)

Salidas (Puertos C y D):

- RC0 -> LED_VERDE (Indicador de acceso permitido / confirmación)
- RC1 -> LED_ROJO (Indicador de acceso denegado / sistema bloqueado)
- RD0 - RD6 -> Conexión a Display de 7 Segmentos (Cátodo Común) mediante resistencias limitadoras de 330Ω.

Diagrama de conexiones



Explicación del manejo de entradas y salidas

El microcontrolador fue configurado para operar con su oscilador interno a una frecuencia de 1 MHz (FOSC = INTIO67).

Configuración de Registros: Para asegurar el correcto funcionamiento digital, se limpiaron los registros analógicos (ANSELB, ANSELC, ANSELD = 0x00), forzando a los pines a trabajar

exclusivamente con lógica discreta (0 y 1). Posteriormente, mediante los registros TRIS, se definió la dirección de los datos:

- TRISB = 0xFF: Configura todo el Puerto B como entradas para leer los estados lógicos de los pulsadores e interruptores.
- TRISC = 0xFC (Bits 0 y 1 en '0'): Configura RC0 y RC1 como salidas para controlar los LEDs indicadores.
- TRISD = 0x00: Configura todo el Puerto D como salidas para enviar los datos de los segmentos al display.

Manejo de Hardware y Software

Las entradas físicas (botones) utilizan resistencias *Pull-Down* de 10kΩ conectadas a tierra para garantizar un estado lógico '0' cuando no son presionadas, evitando lecturas flotantes. Para mitigar el ruido mecánico que se genera al presionar un botón (rebote), se implementó una técnica de **antirrebote por software**. Esta técnica consiste en leer el estado lógico del pin, aplicar un retardo de estabilización (__delay_ms(20)) y volver a verificar el estado del pin. Adicionalmente, se incluyó un bucle while(boton == 1); que detiene el flujo de esa instrucción específica hasta que el usuario suelta el pulsador, garantizando que el microcontrolador registre un solo evento por cada pulsación física, previniendo incrementos descontrolados en el display.

Finalmente, los retardos para la visualización de los LEDs (2 segundos) se segmentaron en bucles de retardos menores (for con pausas de 100ms) para evitar que el procesador quede completamente bloqueado por un tiempo prolongado, permitiendo una ejecución más fluida de la máquina de estados.

Código fuente

```
#pragma config FOSC = INTIO67   // Oscilador Interno
#pragma config WDTEN = OFF      // Watchdog Timer apagado
#pragma config LVP = OFF        // Low Voltage Programming apagado
// ------------------------------------------------
#include <xc.h>
#define _XTAL_FREQ 1000000
```

```
#define BTN_UP     PORTBbits.RB0
#define BTN_OK     PORTBbits.RB1
#define BTN_CLR    PORTBbits.RB2
#define SW_ADMIN   PORTBbits.RB3
#define SW_ENABLE  PORTBbits.RB4

#define LED_VERDE  LATCbits.LATC0
#define LED_ROJO   LATCbits.LATC1

const unsigned char display_7seg[10] = {
   0x3F,
   0x06,
   0x5B,
   0x4F,
   0x66,
   0x6D,
   0x7D,
   0x07,
   0x7F,
   0x67
};

unsigned char numero_actual = 0;
unsigned char codigo_guardado = 0;

void init_GPIO(void) {
   ANSELB = 0x00;
   ANSELC = 0x00;
   ANSELD = 0x00;
```

```
    TRISB = 0xFF;

    TRISCbits.TRISC0 = 0;
    TRISCbits.TRISC1 = 0;
    TRISD = 0x00;

    LATC = 0x00;
    LATD = display_7seg[0];
}

void actualizar_display(unsigned char numero) {
    LATD = display_7seg[numero];
}

void main(void) {
    OSCCON = 0x30;
    init_GPIO();

    while(1) {
        if (SW_ENABLE == 0) {
            LED_ROJO = 1;
            LED_VERDE = 0;
        } else {
            LED_ROJO = 0;
        }

        if (BTN_UP == 1) {
            __delay_ms(20);
            if (BTN_UP == 1) {
                numero_actual++;
```

```
      if (numero_actual > 9) numero_actual = 0;
      actualizar_display(numero_actual);
      while(BTN_UP == 1);
    }
  }

  if (BTN_CLR == 1 && SW_ADMIN == 0) {
    __delay_ms(20);
    if (BTN_CLR == 1) {
      numero_actual = 0;
      actualizar_display(numero_actual);
      while(BTN_CLR == 1);
    }
  }

  // --- 4. Léctura de BTN_OK ---
  if (BTN_OK == 1 && SW_ENABLE == 1) {
    __delay_ms(20); // Antirrebote
    if (BTN_OK == 1) {

      if (SW_ADMIN == 0) {
        if (numero_actual == codigo_guardado) {
          LED_VERDE = 1;
          for(int i=0; i<20; i++) __delay_ms(100);
          LED_VERDE = 0;
        } else {
          LED_ROJO = 1;
          for(int i=0; i<20; i++) __delay_ms(100);
          LED_ROJO = 0;
        }
```

```
            }
            else if (SW_ADMIN == 1) {
               codigo_guardado = numero_actual;

               LED_VERDE = 1; __delay_ms(250); LED_VERDE = 0; __delay_ms(250);
               LED_VERDE = 1; __delay_ms(250); LED_VERDE = 0;
            }

            while(BTN_OK == 1);
         }
      }
   }
}
```